

Content Table:

#1: API Introduction.....	02
#2: Fast-API Philosophy.....	06
#3: HTTP Methods in Fast-API.....	12
#4: Path & Query Params in Fast-API.....	16
(Path Function)	
(HTTP Status Code)	
(HTTPException Function)	
(Query Function)	
#5: Pydantic.....	21
#6: POST Request.....	22
#7: PUT & DELETE Request.....	24
#8: Patient Management API.....	25
#9: Fast-API & ML Model.....	26
#10: How to Dockerize a FastAPI API Application.....	27

1: API Introduction

What is an API?

APIs are mechanisms that enable two software components—such as the frontend and backend of an application—to communicate with each other using a defined set of rules, protocols, and data formats.



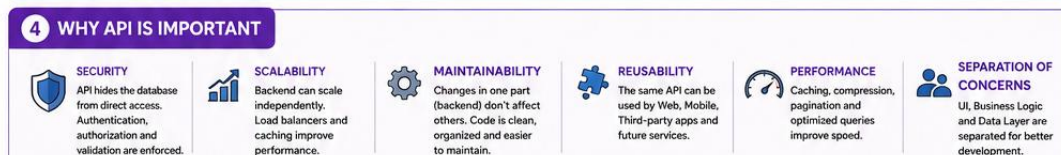
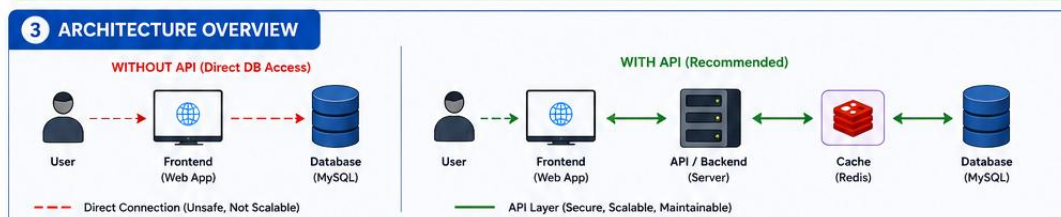
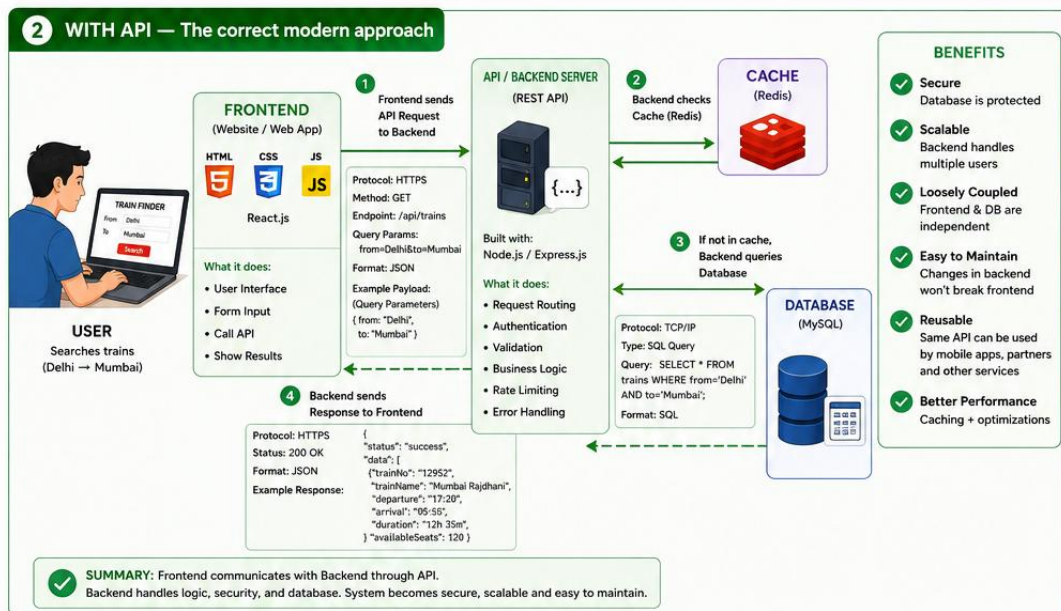
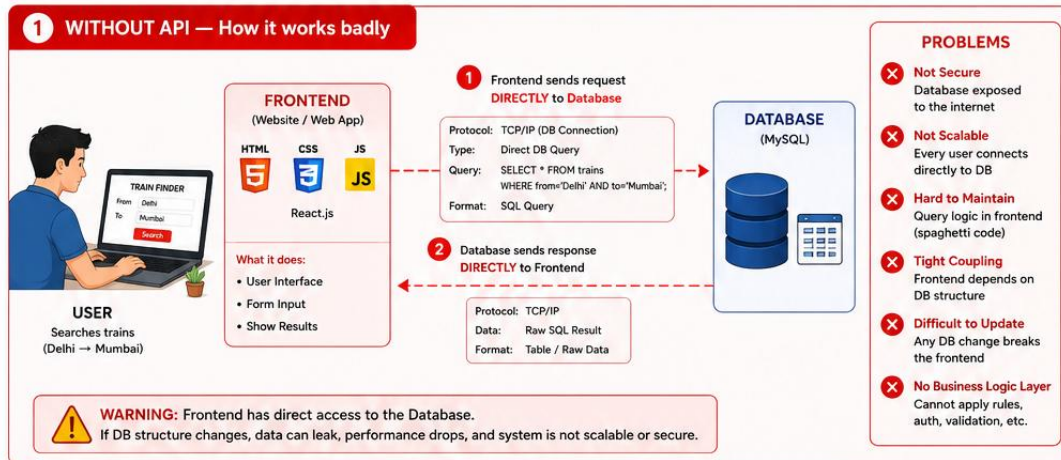
Analogy



NEED OF APIS:

EXAMPLE APP: TRAIN FINDER

User searches trains between two stations (e.g., Delhi to Mumbai) and gets the list of available trains.



● **Without API = Monolithic Architecture**

Frontend directly communicates with the Database.
Everything is tightly coupled into one large system.

Flow: Frontend → Database

● **With API = Microservice Architecture**

Frontend communicates with Backend through APIs.
Each service works independently and is loosely coupled.

Flow: Frontend → API → Backend → Database

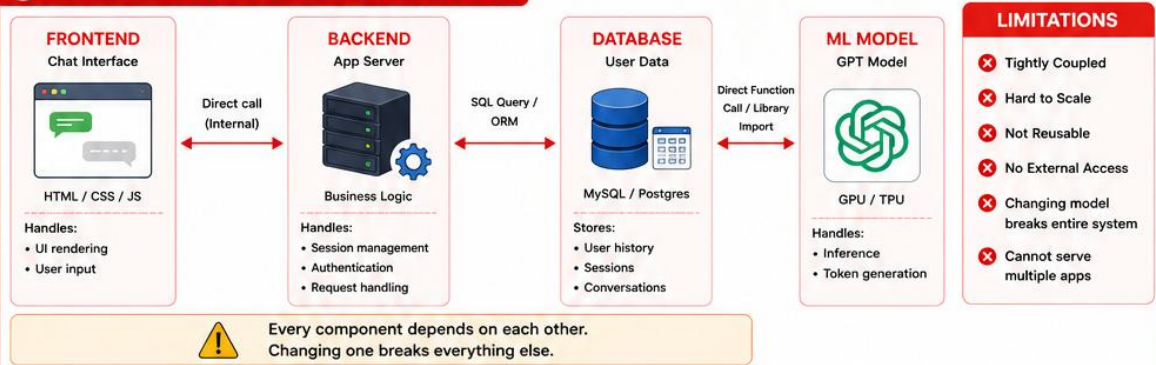
YouTube: Monolith and Microservice Architecture

(must watch)

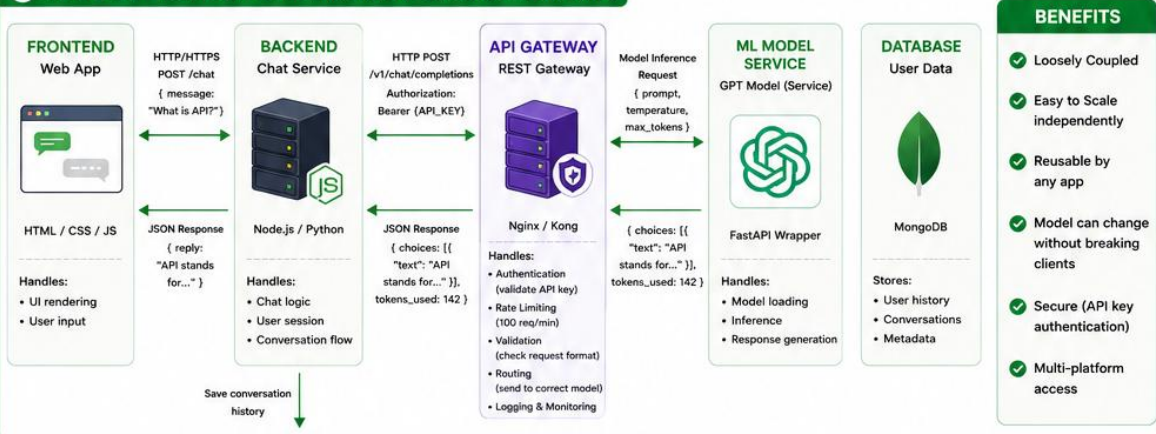
API in ML Perspective – ChatGPT Example

How APIs make ML models accessible, scalable and reusable

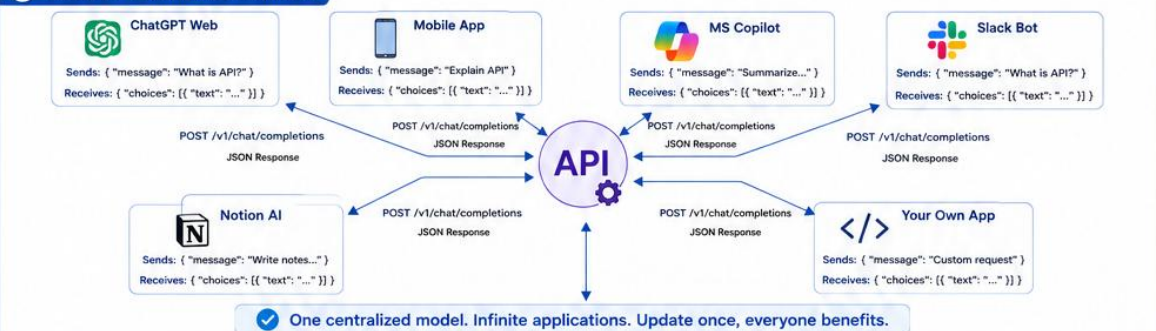
1 Before API – Tightly Coupled (Monolithic)



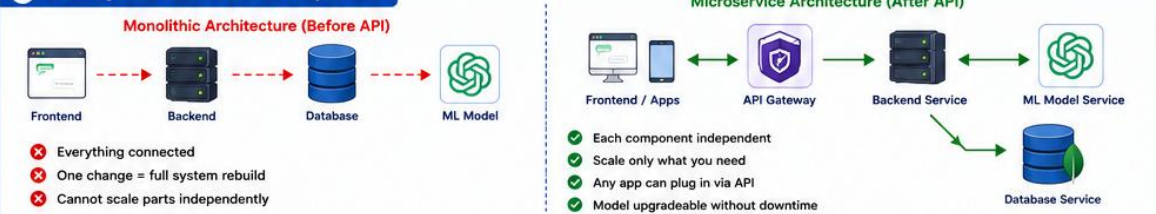
2 After API – Decoupled Architecture (Scalable & Reusable)



3 One API → Many Applications



4 Side-by-side Architecture Comparison



5 Key Takeaway

API acts as a bridge between any application and the ML model. Apps never touch the model directly — they speak through the API. This makes the system:

Scalable • Secure • Flexible • Future-ready



2: Fast-API Philosophy



FastAPI is a modern, high-performance web framework for building APIs with Python



Starlette manages how your API receives requests and sends back responses.



Pydantic is used to check if the data coming into your API is correct and in the right format.

Starlette and Pydantic are Python libraries

FastAPI – Modern Python Web Framework

Build fast, production-ready APIs with Python in minutes

1 WHAT IS FASTAPI?

FastAPI is a modern, high-performance Python web framework for building APIs based on standard Python type hints.

⚡ Fast 🔒 Secure 📄 Auto Docs

Created by: Sebastián Ramírez (2018)

</> Language: Python 3.7+

Based on: Starlette + Pydantic

⚡ Style: Async-first (ASGI)

License: MIT (Open Source)

GitHub Stars: 70,000+ ★

MIT

Requests per second (higher = better)

FastAPI	3,400 req/s
Flask	800 req/s
Django	600 req/s
Express (Node.js)	3,200 req/s

FastAPI matches Node.js/Go speed in Python

2 WHAT MAKES FASTAPI SPECIAL?

High Performance

- Built on Starlette (ASGI framework)
- Async/await support out of the box
- Non-blocking I/O operations
- Comparable to NodeJS and Go

Automatic Docs

- Swagger UI → /docs (interactive)
- ReDoc → /redoc (clean readable)
- Generated from your code automatically
- Zero extra configuration needed

Type Safety

- Uses Python type hints
- Pydantic validates all data
- Auto request/response validation
- Catches errors before runtime

Built-in Security

- OAuth2 support
- JWT token authentication
- API Key authentication
- HTTPS enforcement ready

3 REQUEST LIFECYCLE INSIDE FASTAPI

CLIENT → MIDDLEWARE → ROUTER → ENDPOINT → PYDANTIC → RESPONSE

Browser/ App → HTTP Request POST /predict {data: {...}} → Auth check ✓ Rate limit check ✓ CORS check ✓ → Route matched: /predict Method: POST Handler: predict() → async def predict(item: Item): result = ml_model.predict(item.data) return {"prediction": result} → Raw input data → Pydantic model validates & parses → 200 OK { "prediction": "cat", "confidence": 0.97 }

4 FASTAPI AS AN ML MODEL SERVER

Most popular way to serve ML models in production

ML MODEL → FASTAPI SERVER → CLIENTS

TensorFlow → POST /predict

PyTorch → POST /classify

Scikit-learn → POST /recommend

HuggingFace → GET /health

FASTAPI SERVER

- Loads model on startup
- Validates input with Pydantic
- Runs model inference
- Returns JSON prediction
- Handles 1000s of requests async

CLIENTS

- Web App
- Mobile App
- Other APIs
- Dashboard

```

from fastapi import FastAPI
from pydantic import BaseModel
import numpy as np

app = FastAPI()

class InputData(BaseModel):
    features: list[float]

@app.post("/predict")
async def predict(data: InputData):
    prediction = model.predict([data.features])
    return {"prediction": float(prediction[0])}

```

5 FASTAPI VS FLASK VS DJANGO REST

Feature	FastAPI ✓	Flask ⚠	Django REST ⚠
Performance	⚡ Very High	Medium	Medium
Auto Documentation	✓ Built-in	✗ Manual	⚠ Partial
Type Validation	✓ Pydantic	✗ Manual	⚠ Serializers
Async Support	✓ Native	⚠ Plugin	⚠ Limited
Learning Curve	Easy	Easy	Steep
ML/AI Integration	✓ Best fit	⚠ OK	✗ Heavy
Production Ready	✓ Yes	⚠ Extra	✓ Yes
Auto API Schema	✓ OpenAPI	✗ No	⚠ Manual

6 FASTAPI FULL PRODUCTION STACK

CLIENT → GATEWAY (REVERSE PROXY) → FASTAPI APP → WORKERS (ASGI SERVER) → TASKS (BACKGROUND JOBS) → DATABASE → CACHE → MONITORING & LOGS → DEPLOYMENT

React / Vue / Mobile / Postman

Nginx / Traefik

Your FastAPI application

Uvicorn + Gunicorn

Celery + Redis

PostgreSQL / MySQL / MongoDB

Redis

Prometheus + Grafana / ELK

Docker + Kubernetes / Cloud

Scalable
Secure
Production Ready

Why Developers ❤️ FastAPI?

🚀 Blazing Fast Performance </> Less Code More Results 🛡️ Built-in Docs & Validation 🔒 Secure by Default > Perfect for ML / AI APIs ⚡ FastAPI Build APIs the modern way!

• Main Philosophy of Fast-API:

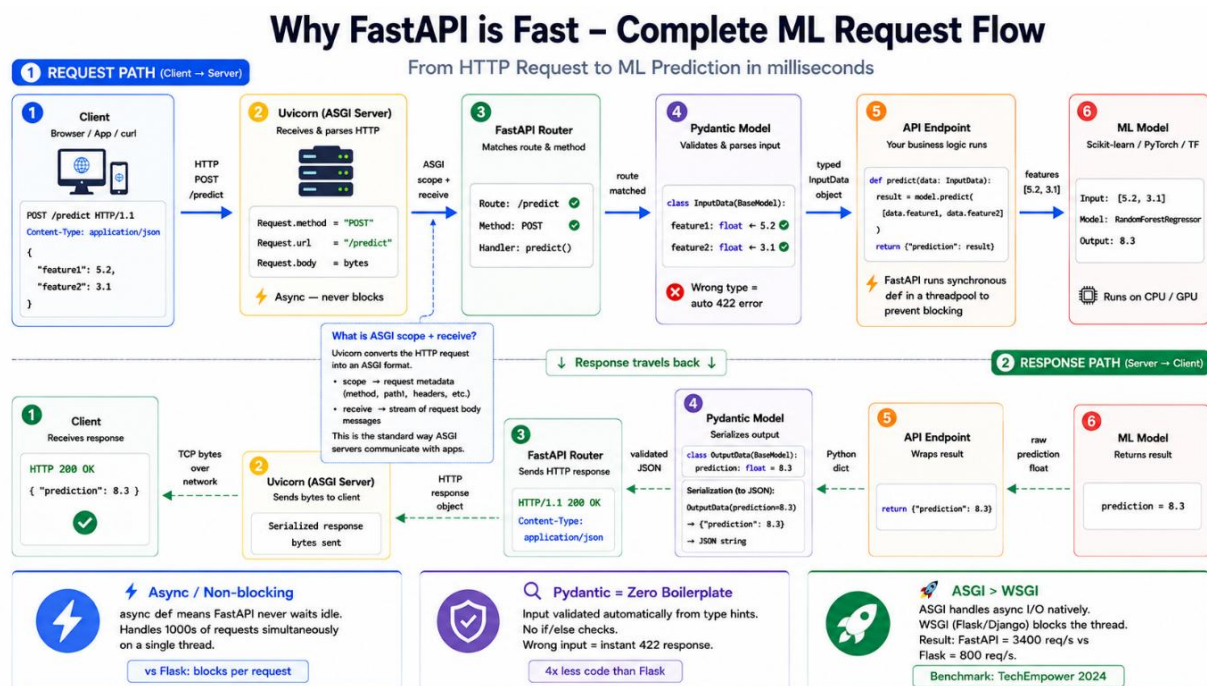


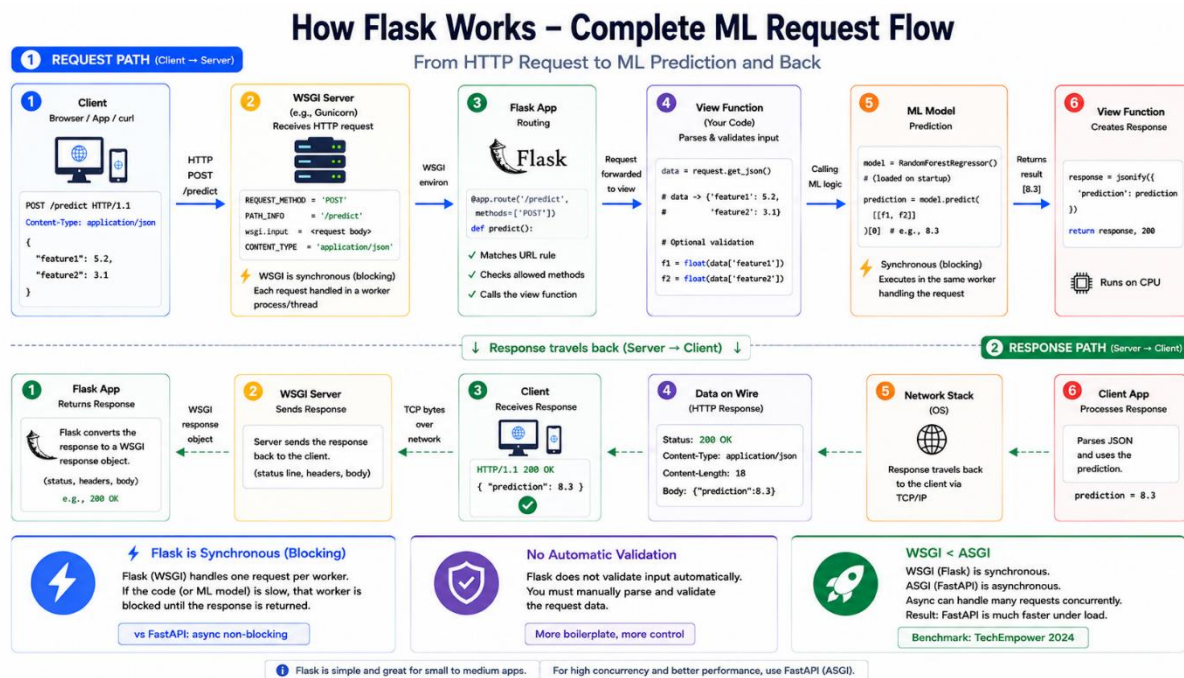
Fast to Run



Fast to Code

#1: Why Fast to Run??





(Just by reading these two images, you will understand why Fast-API is better.)

#2: Why Fast-API is Fast to Code??

1. Automatic input Validation
2. Auto – Generated interactive Documentation
3. Seamless integration with Modern Ecosystem(ML/DL libraries, OAuth, JWT, SQL Alchemy, Docker, Kubernetes etc.)

Let's Create Hello World Fast-API:

1. Create a virtual environment
2. Activate environment
3. Pip install fastapi, uvicorn, pydantic
4. Main.py

```
from fastapi import FastAPI

# Create an object (instance) of FastAPI
# This object will be our main application
app = FastAPI()

# -----
# Route: "/"
# -----
# @app.get("/") means:
# When someone visits the homepage URL using a GET request,
# run the function written below it.
#
# Example:
# http://127.0.0.1:8000/
#
# GET request is mainly used to fetch/read data.
@app.get("/")

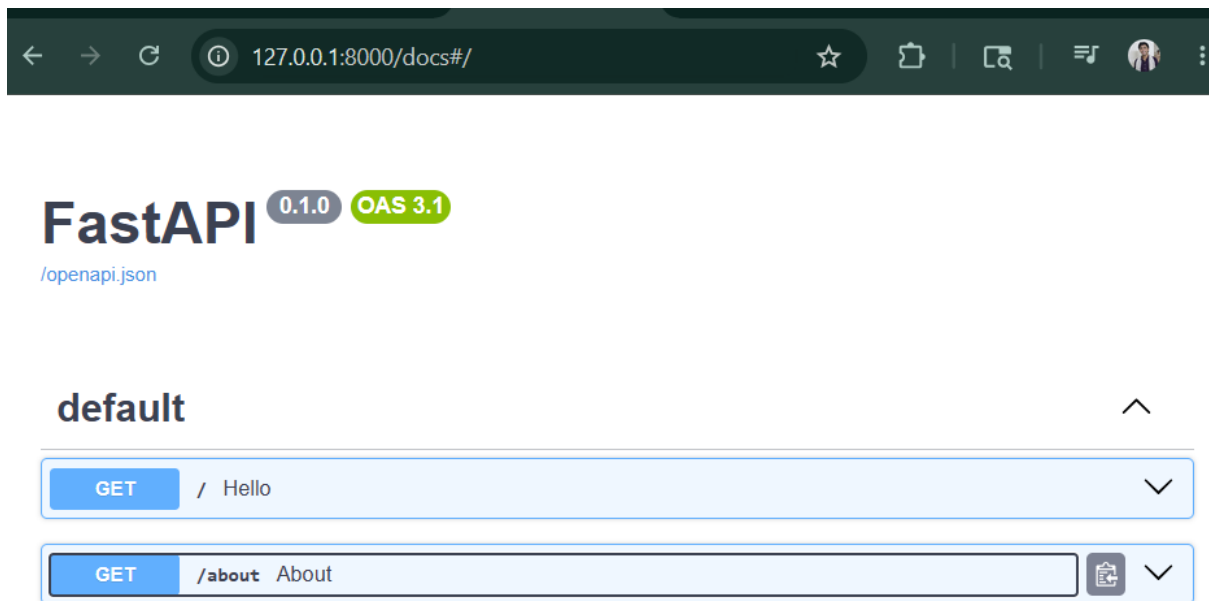
# Function that will run when the "/" route is accessed
def hello():

    # Return data in dictionary format
    # FastAPI automatically converts this dictionary into JSON
    #
    # Output in browser/API response:
    # {
    #     "message": "hello world"
    # }
    return {"message": "hello world"}
```

5. Run code: `uvicorn main:app --reload`

You will get the link in terminal simple open it and check.

Auto documentation created by the FastAPI:



3: HTTP Methods in Fast-API

1. Static VS Dynamic Software:

STATIC vs DYNAMIC SOFTWARE

Software can be broadly classified into two types: **Static Software** and **Dynamic Software**.

STATIC SOFTWARE

Static software **does not change** while running. Once compiled, its structure and behavior remain fixed.

EXAMPLE: CALENDAR



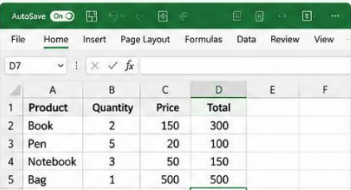
FIXED
Does not change on its own.

Behavior	Fixed and does not change while running.
Data Handling	Does not depend on user input to change the content.
Changes	Any change requires recompilation or reinstalling.
Memory Usage	Usually lower and constant.
Real-life Example	Calendar application The calendar shows fixed dates. It doesn't change unless the program is updated.

DYNAMIC SOFTWARE

Dynamic software **can change** while running. Its behavior can vary based on user input and real-time data.

EXAMPLE: EXCEL



CHANGEABLE
Changes based on user input and real-time data.

Behavior	Can change while running based on user input or real-time data.
Data Handling	Heavily depends on user input, calculations, and external data.
Changes	No need to recompile; changes happen instantly.
Memory Usage	Usually higher and varies with operations.
Real-life Example	Excel application Values change instantly when we enter data, apply formulas, or edit anything.

SUMMARY

Calendar is Static – It shows fixed information.
Excel is Dynamic – It changes based on what we do.

- Let's talk about Dynamic Software:

In how many ways can you interact with the dynamic software??

Only in 4 ways (CRUD)

C – create

R - retrieve

U – update

D – Delete

Aryan Chaudhari

Is this true for the website as well??

A website is also software, but not installed on your device; it's installed on a server. The server is also known as the client.

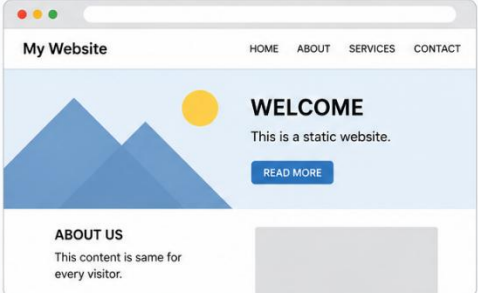
Client and server use HTTP Method to communicate

2. Static VS Dynamic Websites:

TWO TYPES OF WEBSITES

1. STATIC WEBSITE

Static websites show the same content to every user. The content is **fixed**.



ABOUT US
This content is same for every visitor.

Content is fixed
Same content for all users

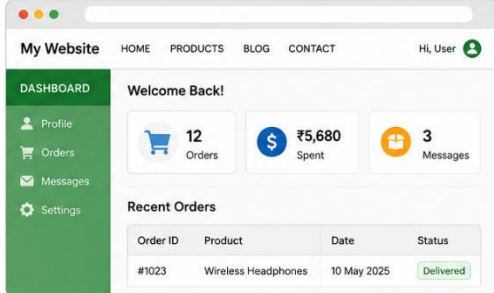
Built with HTML, CSS, JavaScript
(No server-side processing)

Faster and more secure
Less database load

EXAMPLE:
Portfolio Website, Landing Page, Brochure Website

2. DYNAMIC WEBSITE

Dynamic websites show different content to different users. The content **changes**.



Recent Orders

Order ID	Product	Date	Status
#1023	Wireless Headphones	10 May 2025	Delivered

Content is changeable
Different content for different users

Built with HTML, CSS, JavaScript + Server-side
(e.g., PHP, Python, Node.js) and Database

More interactive and powerful
Uses database and server processing

EXAMPLE:
E-commerce Website, Social Media, News Website

SUMMARY:

Static websites are fixed and simple.

Dynamic websites are changeable and interactive.

- **Let's talk about Dynamic Websites:**

In how many ways can you interact with the dynamic Websites??

Only in 4 ways (CRUD)

C → create

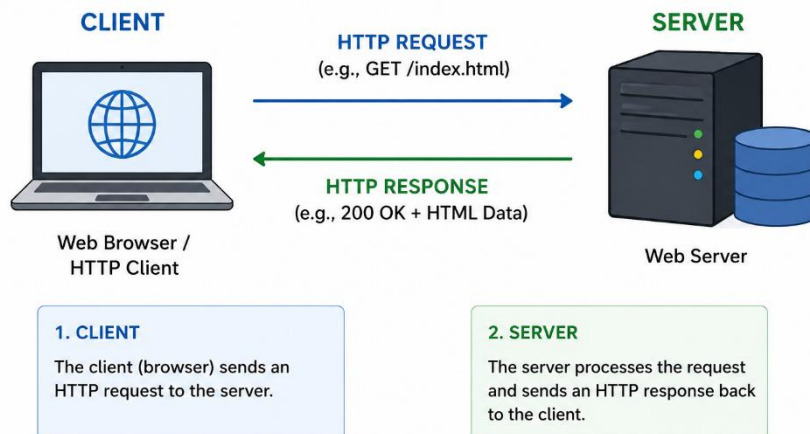
R → retrieve

U → update

D → Delete

(But this happened using the internet.)

HTTP CLIENT – SERVER MODEL



When we communicate using HTTP, we must specify the type of action we want to perform by using an HTTP method (verb) such as GET, POST, PUT, PATCH, or DELETE. These verbs tell the server what kind of operation the client wants to do.

Examples

- **GET** → Retrieve data from the server
- **POST** → Send/create new data on the server
- **PUT** → Update existing data completely
- **PATCH** → Update part of the existing data
- **DELETE** → Remove data from the server

4: Path & Query params in Fast-API

1. Path VS Query Parameter:

PATH PARAMETER vs QUERY PARAMETER

Both are used to send data from client to server, but they serve **different purposes**.

PATH PARAMETER

WHAT IS IT?
A path parameter is a part of the URL path. Used to identify a specific resource.

ANALOGY
 Imagine you are giving an exact address to reach a friend's house.
The house number (part of the address) uniquely identifies the house.

SYNTAX (FastAPI)

```
@app.get("/users/{user_id}")
def get_user(user_id: int):
    return {"user_id": user_id}
```

DATA TYPES (example from DB)

- int -> Numeric IDs
- str -> UUIDs, slugs
- float -> Prices, ratings
- bool -> true/false
- datetime -> Dates, timestamps

EXAMPLE REQUEST
GET `http://127.0.0.1:8000/users/101`
Path Parameter (user_id: int)

EXAMPLE RESPONSE

```
{
  "id": 101,
  "name": "Ananya Sharma",
  "email": "ananya@email.com"
}
```


Returns the specific user with ID = 101

WHEN TO USE?

- To retrieve, update or delete a specific resource
- When the resource identity is required
- Example: `/users/101, /orders/5501, /products/ABC123`

KEY POINTS

- Part of the URL path (cannot be optional)
- Usually required
- Identifies a single resource
- Makes the API URL clean and RESTful

QUERY PARAMETER

WHAT IS IT?
A query parameter is added after '?' in the URL. Used for filtering, searching, sorting, pagination, etc.

ANALOGY
 Imagine you are telling the shop what kind of items you want using filters (like category, price, brand, etc.).
These are not the address, but extra instructions to filter results.

SYNTAX (FastAPI)

```
@app.get("/products/")
def get_products(
    category: str | None = None,
    min_price: float | None = None,
    in_stock: bool | None = None,
    limit: int = 10,
    offset: int = 0
):
    pass
```

COMMON DATA TYPES

- str -> category, search text
- int -> price, limit, page
- float -> min_price, max_price
- bool -> in_stock, is_active
- datetime -> start_date, end_date
- list[str] -> tags, ids (comma separated)

EXAMPLE REQUEST
GET `http://127.0.0.1:8000/products?category=books&min_price=100&in_stock=true&limit=5&offset=0`
 ↑ Query Param (str) ↑ Query Param (float) ↑ Query Param (bool) ↑ Query Param (int) ↑ Query Param (int)

EXAMPLE RESPONSE

```
{
  "total": 125,
  "limit": 5,
  "offset": 0,
  "products": [
    {
      "id": 3,
      "name": "FastAPI Book",
      "price": 199.0,
      "in_stock": true,
      "tags": ["python", "web"]
    },
    {
      "id": 2,
      "name": "Python Basics",
      "price": 150.0,
      "in_stock": true,
      "tags": ["python", "web"]
    }
  ]
}
```


Returns a list of products filtered by the query parameters

WHEN TO USE?

- To filter, search, sort, paginate data
- When the resource is the same but data changes
- Example: `/products?category=books&min_price=100`

KEY POINTS

- Added after '?' in the URL
- Usually optional (can have default values)
- Used to modify or filter the result set

BOTH TOGETHER (Example)
GET `http://127.0.0.1:8000/users/101/orders?status=delivered&limit=5`
 Path Parameter (user_id: int) Query Param (status: str) Query Param (limit: int)

Meaning: Get orders of user with ID 101, only those with status = delivered, limit = 5

EXAMPLE DATABASE (for reference)

USERS			ORDERS			
id (int)	name (str)	email (str)	id (int)	user_id (int)	status (str)	total (float)
101	Ananya Sharma	ananya@email.com	5001	101	delivered	1200.50
102	Ravi Mehta	ravi@email.com	5002	101	pending	950.00
103	Sneha Kulkarni	sneha@email.com	5003	102	delivered	750.25

QUICK COMPARISON

Feature	Path Parameter	Query Parameter
Location	In the URL path	After '?' in the URL
Purpose	Identify resource	Filter / Search / Modify
Required	Usually required	Usually optional
Example	<code>/users/101</code>	<code>/products?category=books</code>
Can be Multiple?	Yes (multiple in path)	Yes (multiple as key=value)
Affects Resource?	Changes the resource	Modifies the result set

■ Required ■ Optional

★ **NOTE:** Follow REST API best practices: Use nouns in URL, path params for resource identity, query params for filtering and options.

2. Path Function:

Path Function in FastAPI

The Path() function in FastAPI is used to add:

- metadata
- validation rules
- documentation hints

for **path parameters** in API endpoints.

It helps make APIs:

- more secure
- well documented
- easier to validate



FastAPI Path() Function – Common Parameters {/}

Path() is used to add **validation rules**, **metadata** and **documentations hints** for **path parameters** in FastAPI.

Parameter	Purpose	Example in Path()	Sample Valid Value
 title	Short title of the parameter shown in docs.	Path(..., title="User ID")	User ID
 description	Explain the parameter in detail.	Path(..., description="Unique ID of the user")	Unique ID of the user
 example	Example value for the parameter.	Path(..., example=101)	101
 gt	Value must be greater than this.	Path(..., gt=0)	1, 10, 100
 ge	Value must be greater than or equal to this.	Path(..., ge=0)	0, 1, 10, 100
 lt	Value must be less than this.	Path(..., lt=100)	1, 10, 99
 le	Value must be less than or equal to this.	Path(..., le=100)	0, 1, 10, 100
 min_length	Minimum length required (for string types).	Path(..., min_length=3)	"abc", "FastAPI"
 max_length	Maximum length allowed (for string types).	Path(..., max_length=20)	"HelloWorld", "FastAPI_Rocks"
 regex / pattern	Match the value with the specified regular expression.	Path(..., pattern=r"^[A-Z]{3}\d{4}\$")	"ABC1234", "XYZ9999"


Quick Example

```

from fastapi import FastAPI, Path
app = FastAPI()

@app.get("/users/{user_id}")
def get_user(user_id: int = Path(gt=0, le=1000, title="User ID", example=101)):
    return {"user_id": user_id

```



Why Use These Parameters?

- ✔ Ensure data validation
- ✔ Improve API documentation (Swagger UI)
- ✔ Make APIs more reliable and user-friendly

★ **Note:** Use these parameters to enforce rules on path parameters and provide clear documentation for API users.

3. HTTP Status Codes:

- Before starting the next topic, let's discuss HTTP Status Codes:

⇒ HTTP STATUS CODES ⇒

HTTP Status Codes are 3-digit numbers returned by a server in response to a client's request.

1xx – INFORMATIONAL

 The request was received and the process is continuing.

Code	Meaning
100	Continue
101	Switching Protocols

2xx – SUCCESS

 The request was successfully received, understood, and accepted.

Code	Meaning
200	OK
201	Created
204	No Content

3xx – REDIRECTION

 Further action is needed to complete the request.

Code	Meaning
301	Moved Permanently
302	Found (Temporary)
304	Not Modified

4xx – CLIENT ERROR

 The request has a problem due to something on the client side.

Code	Meaning
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
429	Too Many Requests

5xx – SERVER ERROR

 The server failed to fulfill a valid request.

Code	Meaning
500	Internal Server Error
502	Bad Gateway
503	Service Unavailable
504	Gateway Timeout

QUICK EXAMPLES

200 OK	– The page or data is returned successfully.
404 Not Found	– The requested resource does not exist.
500 Internal Server Error	– Something went wrong on the server.
401 Unauthorized	– You need to login or provide valid credentials.
302 Found	– The resource is temporarily available at another URL.

SUMMARY

	1xx Informational Request received	2xx Success Request successful	3xx Redirection Go to another location	4xx Client Error Problem in your request	5xx Server Error Problem on server
---	--	--	--	--	--

★ Remember: These codes help browsers, apps, and developers understand what happened with a request. 😊

- HTTPException Class in FastAPI

HTTPException is used in FastAPI to return:

- custom error messages
- HTTP status codes
- validation/error responses

to the client.

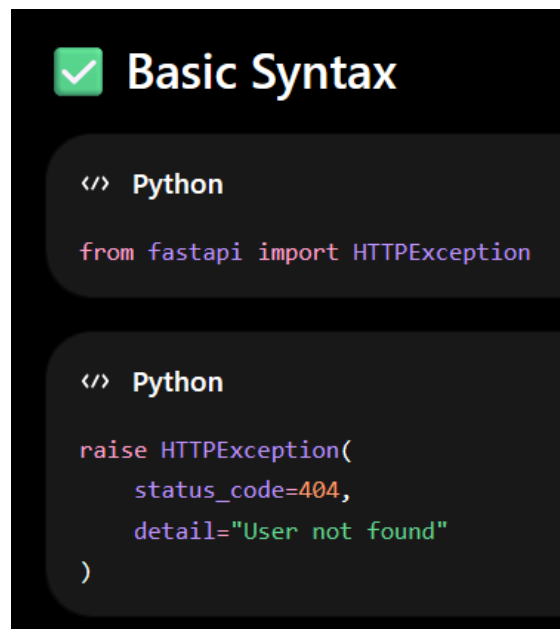
It helps stop the request and send an error response immediately.

- **HTTPException Function:**

Suppose:

- user not found
- wrong ID
- unauthorized access
- invalid data

Instead of crashing the API, we use HTTPException.



4. Query Function:

Query Function in FastAPI

The Query() function in FastAPI is used to:

- add validation
- add metadata
- set default values
- document query parameters

It works for **query parameters** just like Path() works for path parameters.



FASTAPI Query() FUNCTION



Query() is used to add **validation**, **metadata** and documentation for query parameters in FastAPI.

WHAT IS QUERY()?

Query() is used to define query parameters in the URL.

`http://127.0.0.1:8000/products?category=mobile&limit=10`

Path: `/products`
Query Parameters: `?category=mobile&limit=10`

- ✓ Used for filtering, searching, sorting, etc.
- ✓ Helps in data validation
- ✓ Adds docs in Swagger UI
- ✓ Supports default values

BASIC SYNTAX

```
from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/products")
def get_products(
    category: str = Query(...)
):
    return {"category": category}
```

- Query(...) → Required query parameter
- Query(default) → Optional with default value

EXAMPLE URL

`http://127.0.0.1:8000/products?category=mobile&limit=10`

RESPONSE

```
{
  "category": "mobile",
  "limit": 10
}
```

COMMON PARAMETERS OF Query()

Parameter	Purpose	Example
<code>title</code>	Short title of parameter	<code>Query(..., title="Category")</code>
<code>description</code>	Explain parameter in detail	<code>Query(..., description="Product category")</code>
<code>example</code>	Example value	<code>Query(..., example="mobile")</code>
<code>gt</code>	Greater than	<code>Query(..., gt=0)</code>
<code>ge</code>	Greater than or equal	<code>Query(..., ge=1)</code>
<code>lt</code>	Less than	<code>Query(..., lt=100)</code>
<code>le</code>	Less than or equal	<code>Query(..., le=100)</code>
<code>min_length</code>	Minimum string length	<code>Query(..., min_length=3)</code>
<code>max_length</code>	Maximum string length	<code>Query(..., max_length=20)</code>
<code>regex / pattern</code>	Match specific format	<code>Query(..., pattern="^[A-Z]{3}\$")</code>
<code>default</code>	Default value	<code>Query(10)</code>

EXAMPLE 1: VALIDATION (NUMERIC)

```
@app.get("/items")
def get_items(
    limit: int = Query(10, gt=0, le=100)
):
    return {"limit": limit}
```

Meaning

- Default = 10
- Must be greater than 0
- Must be less than or equal to 100

Example URLs

URL	Response
<code>/items</code>	<code>{ "limit": 10 }</code>
<code>/items?limit=50</code>	<code>{ "limit": 50 }</code>
<code>/items?limit=150</code>	<code>Response: Validation Error</code>

EXAMPLE 2: MULTIPLE QUERY PARAMETERS

```
@app.get("/products")
def get_products(
    category: str = Query(None),
    min_price: int = Query(0),
    limit: int = Query(10)
):
    return {
        "category": category,
        "min_price": min_price,
        "limit": limit
    }
```

Example URL

`/products?category=mobile&min_price=10000&limit=5`

Response

```
{
  "category": "mobile",
  "min_price": 10000,
  "limit": 5
}
```

EXAMPLE 3: STRING VALIDATION

```
@app.get("/users")
def get_user(
    name: str = Query(
        min_length=3,
        max_length=20
    )
):
    return {"name": name}
```

Example URLs

<code>/users?name=Bunny</code>	✓ Valid
<code>/users?name=A1</code>	✗ Error (Too short)
<code>/users?name=ThisIsAReallyLongNameExceedingLimit</code>	✗ Error (Too long)

KEY POINTS

- ✓ Query parameters come after "?" in the URL.
- ✓ Used for filtering, searching, pagination etc.
- ✓ Can be required or optional.
- ✓ Supports default values.
- ✓ Automatic documentation in Swagger UI.

PATH() vs QUERY()

Path()	Query()
Used in URL path	Used after "?" in URL
Identify a resource	Filter / search resource
Mandatory in path	Optional / default
<code>/users/101</code>	<code>/users?limit=10</code>

Remember: Use Query() to make your APIs **flexible, safe, and well-documented!**

5: Pydantic

[\[YouTube: Pydantic Crash Course\]](#)

OR

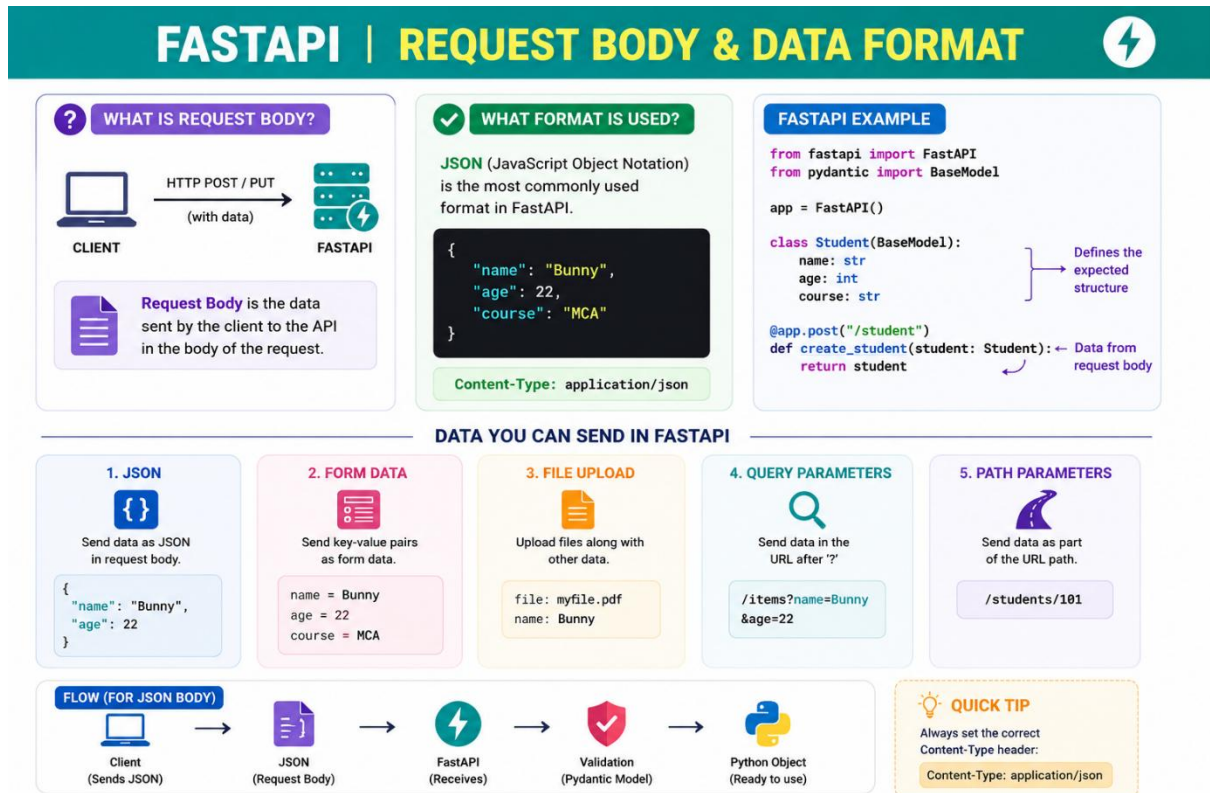
You can check my Pydantic folder.

Path:

DS- AI-ML-JOURNEY → 02-Software Essentials → 04_What is API(FastAPI)→ Pydantic

[\[LINK\]](#)

6: Post Request



[After validation, the data will be updated.]



DATA YOU CAN SEND IN FASTAPI

FastAPI accepts data in different ways depending on your need.

1. JSON (Request Body)



Send structured data in JSON format in the request body.

How it looks (JSON)

```
{
  "name": "Bunny",
  "age": 22,
  "course": "MCA"
}
```

FastAPI code example

```
from pydantic import BaseModel
from fastapi import FastAPI

app = FastAPI()

class Student(BaseModel):
    name: str
    age: int
    course: str

@app.post("/student")
def create_student(student: Student):
    return student
```

Use for: Sending complex or structured data.

2. FORM DATA



Send key-value pairs as form data.

How it looks (Form Data)

```
name = Bunny
age = 22
course = MCA
```

FastAPI code example

```
from fastapi import FastAPI, Form

app = FastAPI()

@app.post("/form")
def create_form_data(
    name: str = Form(...),
    age: int = Form(...),
    course: str = Form(...)
):
    return {"name": name, "age": age, "course": course}
```

Use for: Simple data from HTML forms.

3. FILE UPLOAD



Upload files along with other data.

How it looks (Form Data with File)

```
file: myfile.pdf (file)
name: Bunny (text)
```

FastAPI code example

```
from fastapi import FastAPI, File, UploadFile, Form

app = FastAPI()

@app.post("/upload")
async def upload_file(
    file: UploadFile = File(...),
    name: str = Form(...)
):
    content = await file.read()
    return {"filename": file.filename, "name": name, "size": len(content)}
```

Use for: Uploading files (images, pdf, docs, etc.).

4. QUERY PARAMETERS



Send data in the URL after a question mark.

How it looks (URL)

```
/items?name=Bunny&age=22
```

FastAPI code example

```
from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/items")
def read_items(name: str = Query(...), age: int = Query(...)):
    return {"name": name, "age": age}
```

Use for: Filtering, searching, sorting, pagination, etc.

5. PATH PARAMETERS



Send data as part of the URL path.

How it looks (URL)

```
/students/101
```

FastAPI code example

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/students/{student_id}")
def read_student(student_id: int):
    return {"student_id": student_id}
```

Use for: Identifying specific resources (IDs).

QUICK SUMMARY



JSON
For complex structured data



Form Data
For simple key-value pairs



File Upload
For sending files



Query Params
For optional data in URL



Path Params
For resource identification

IMPORTANT

- For JSON → set header: Content-Type: application/json
- For Form Data / File → Content-Type: multipart/form-data (handled automatically by FastAPI)

7: Put & Delete Request

FASTAPI – PUT vs DELETE

PUT – Update Existing Data

- Used to update an existing resource completely.
- Client sends the updated data in the request body (usually in JSON format).

→ PUT /items/{id} →

(with data)

- Existing data is a Python dictionary
- Data we receive is JSON

Example Request (JSON)

```
{
  "name": "Bunny",
  "age": 23,
  "course": "MCA"
}
```

FastAPI Example

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Student(BaseModel):
    name: str
    age: int
    course: str

@app.put("/students/{student_id}")
def update_student(student_id: int, student: Student):
    # existing data is python dict
    # data we receive is JSON
    # (update logic here)
    return {"message": "Student updated",
            "student_id": student_id,
            "data": student}
```

Example Response (JSON)

```
{
  "message": "Student updated",
  "student_id": 101,
  "data": {
    "name": "Bunny",
    "age": 23,
    "course": "MCA"
  }
}
```

PUT replaces the entire resource with the new data sent in the request body.

DELETE – Remove Existing Data

- Used to delete an existing resource.
- Usually, we send the resource ID in the URL.
- No request body** is required.

→ DELETE /items/{id} →

(no data)

- Existing data is a Python dictionary
- Data we receive is usually the ID in the URL (not JSON body)

Example Request (URL)

```
DELETE /students/101
```

(No request body)

FastAPI Example

```
from fastapi import FastAPI

app = FastAPI()

@app.delete("/students/{student_id}")
def delete_student(student_id: int):
    # existing data is python dict
    # data we receive is ID from URL
    # (delete logic here)
    return {"message": "Student deleted",
            "student_id": student_id}
```

Example Response (JSON)

```
{
  "message": "Student deleted",
  "student_id": 101
}
```

DELETE removes the resource from the server permanently using the ID in the URL.

QUICK COMPARISON

PUT

- Updates an existing resource
- Data sent in JSON body
- Usually: PUT /items/{id}

DELETE

- Deletes an existing resource
- No body required
- Usually: DELETE /items/{id}

24 | Page

8: Patient Management API

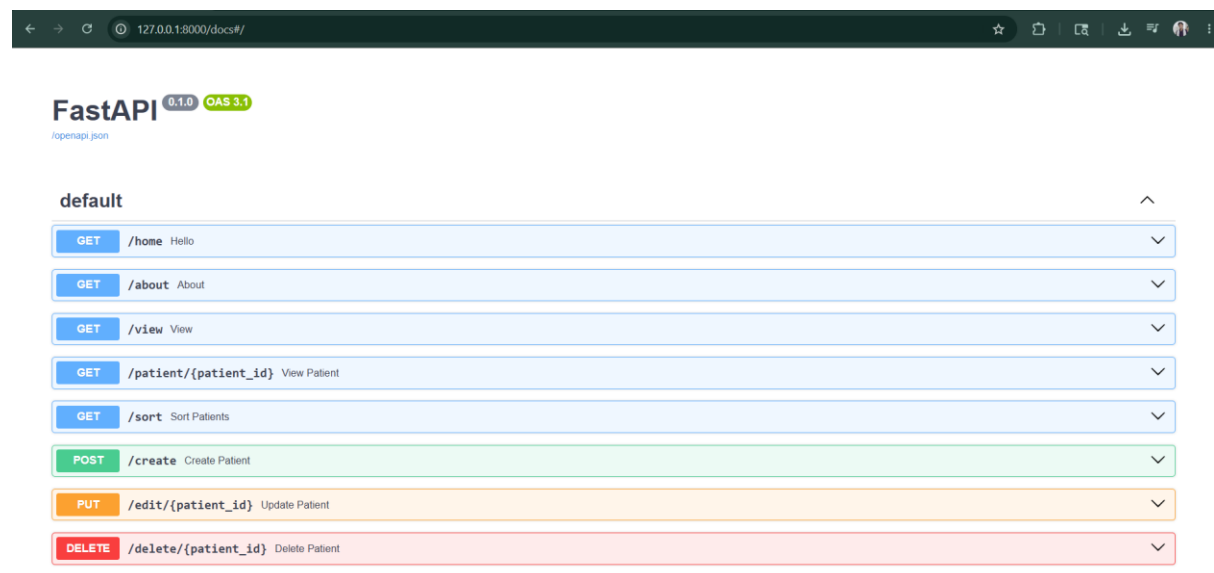
I have created this project, you can check it:

Path:

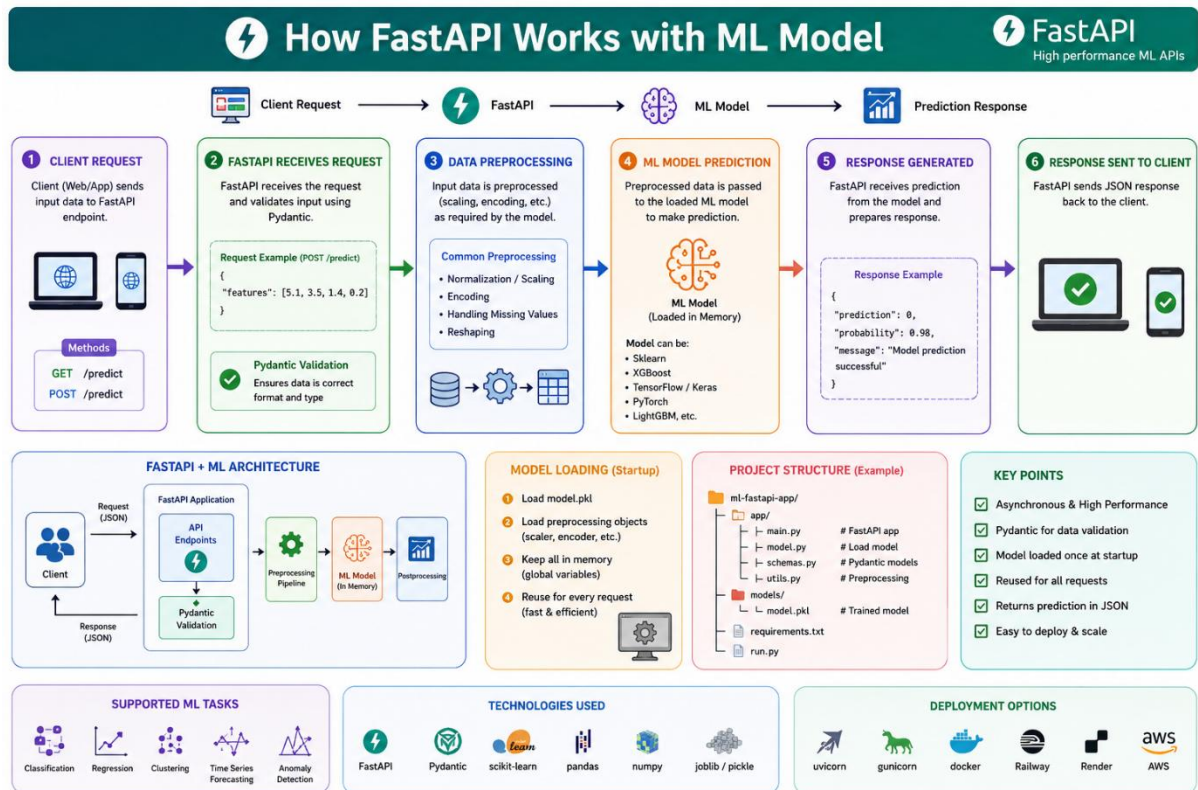
DS- AI-ML-JOURNEY → 02-Software Essentials → 04_What is API(FastAPI)→ Patient management API

[\[LINK\]](#)

DOCS Of my Project:



9: Fast-API & ML Model



I have created an API project on this, you can check it:

Path:

DS- AI-ML-JOURNEY → 02-Software Essentials → 04_What is API(FastAPI)→ Patient management API

10: How to Dockerize a FastAPI API Application

Go and check my Docker folder

First understand basic of Docker.